Advanced Hardware Engineering

Lab 03: GPIO, 7-segment display and PWM IP

Prof. Dr.-Ing. Ali Hayek

05.11.2026

Goals and learning objectives

Goal

Control LEDs, switches, the 7-segment display and your own PWM IP from a C program on the MicroBlaze.

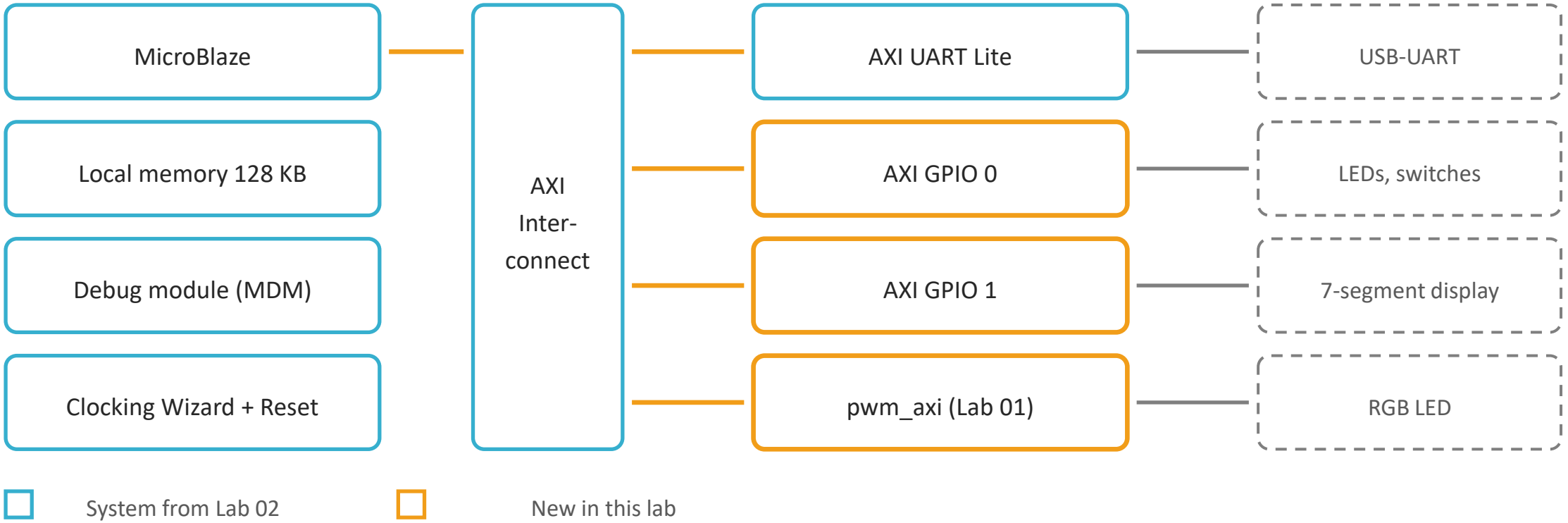
Skills

AXI GPIO configuration, memory-mapped registers, multiplexed displays, adding your own IP to a system, active-low signals.

Deliverables

Screenshot of the block design, utilization summary, C source with comments, and a short demo on the board.

What we will build



Blue: system from Lab 02. Orange: new blocks of this lab.

How software sees the hardware

Block	Offset	Register	Use in this lab
AXI GPIO	0x00	Channel 1 data	Switches (GPIO 0) / segments (GPIO 1)
AXI GPIO	0x04	Channel 1 direction	Not needed (fixed in the design)
AXI GPIO	0x08	Channel 2 data	LEDs (GPIO 0) / digit enables (GPIO 1)
AXI GPIO	0x0C	Channel 2 direction	Not needed (fixed in the design)
pwm_axi	0x00	CTRL	Bit 0 = enable
pwm_axi	0x04, 0x08, 0x0C	DUTY_R, DUTY_G, DUTY_B	8-bit duty of red, green, blue

The board I/O of the Nexys A7

Item	Count	Behavior
LEDs	16	1 = on
Switches	16	Up = 1, down = 0
7-segment display	8 digits	Segments and digit enables are active low; the digits share the segment lines (multiplexing)
RGB LEDs	2	Red, green and blue channel each; dimmed with PWM

Prerequisites and preparation

Item	Details
Lab 02	Your Vivado project with the working MicroBlaze system (a copy is made in step 1.1)
Lab 01	The packaged custom pwm_axi IP folder and its register map
Board files	Digilent board files installed; the board interfaces may be used for LEDs and switches
Tools	Vivado and Vitis of the same release (fill in: [release]); license tier must cover the Artix-7

Task 1: extend the hardware

1. **Prepare the project** – copy of Lab 02, add the IP repository of Lab 01
2. **Add the AXI GPIO blocks and the PWM IP** – configure the channels
3. **Connect and name the external ports** – Connection Automation, Make External
4. **Validate, constrain and build** – XDC file, bitstream
5. **Export the hardware** – new XSA file for Vitis

Hint

In Task 1 you decide which additional hardware exists in the FPGA. In Task 2 you write the software that controls it.

1.1 Prepare the project

1. Open the Lab 02 project in Vivado
2. Select File, Project, Save As and create a copy, e.g. lab03_gpio_pwm
3. Open the block design of the copy
4. Check that the MicroBlaze system and the UART are present

Hint

Make sure you use a copy of your Lab 02 project. A copy keeps your working Lab 02 system intact. If something goes wrong you can always go back to it.

1.2 Add the IP repository with your PWM IP

Add your IP to the design with Add IP. If you cannot find your PWM IP, add the folder of your IP repositories to the project: open Settings, IP, Repository and add the path of your IP repositories:

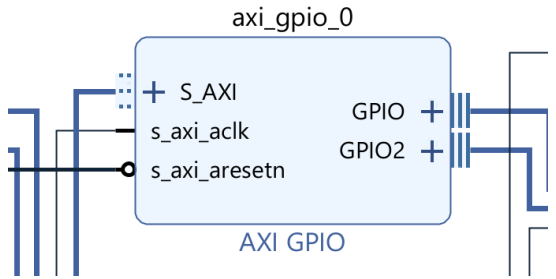
1. Select Settings, IP, Repository
2. Click + and select the folder with the packaged IP from Lab 01
3. Check that Vivado reports one IP found (pwm_axi)
4. Click Apply and OK

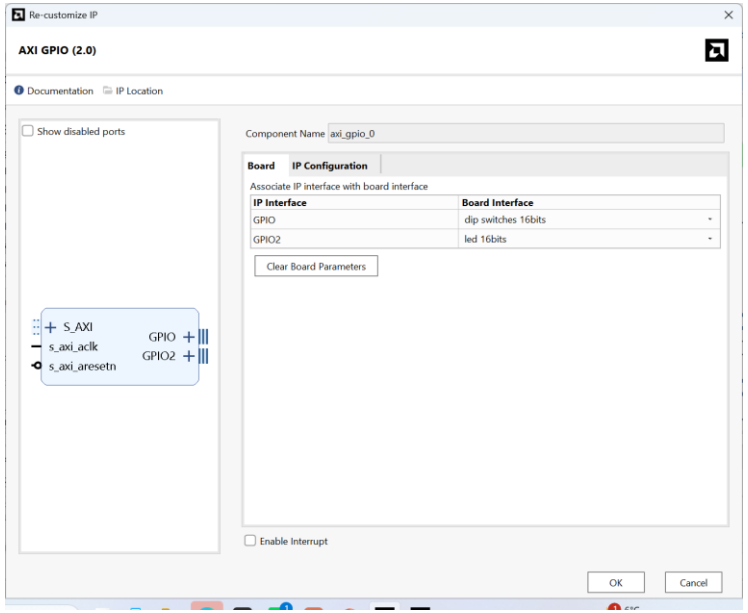
Hint

An IP repository is a folder in which Vivado looks for additional IP. Your packaged PWM appears in the IP catalog next to the AMD IP.

1.3 Add the AXI GPIO for LEDs and switches

1. Press + and search for AXI GPIO, add it
2. Double-click the block to customize it
3. Board tab: GPIO → dip switches 16bits, GPIO2 → led 16bits
4. Without board interfaces (IP Configuration tab): Dual Channel on, channel 1 width 16 All Inputs, channel 2 width 16 All Outputs
5. Click OK

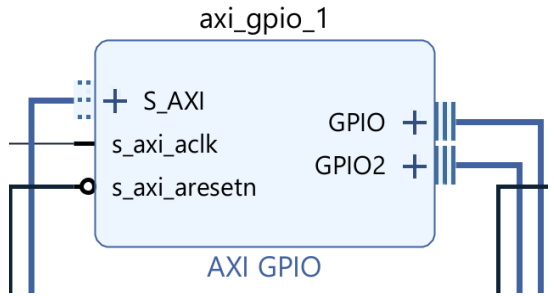


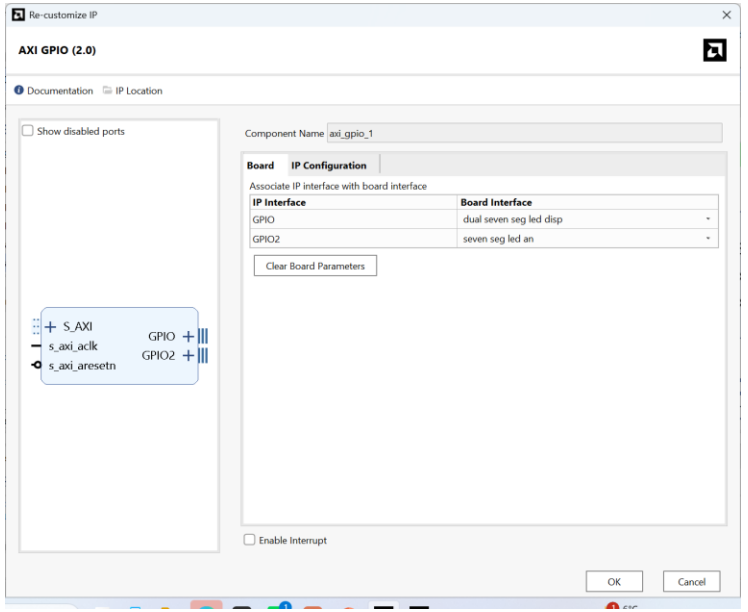


Hint
A GPIO is a bank of parallel pins. One AXI GPIO block has two channels, so one block can serve the 16 LEDs (outputs) and the 16 switches (inputs).

1.4 Add a second AXI GPIO for the 7-segment display

1. Add another AXI GPIO
2. Double-click the block to customize it
3. Board tab: GPIO → dual seven seg led disp (segments CA to CG, DP), GPIO2 → seven seg led en (digit enables AN0 to AN7)
4. Without board interfaces (IP Configuration tab): Dual Channel on, both channels width 8, All Outputs
5. Click OK

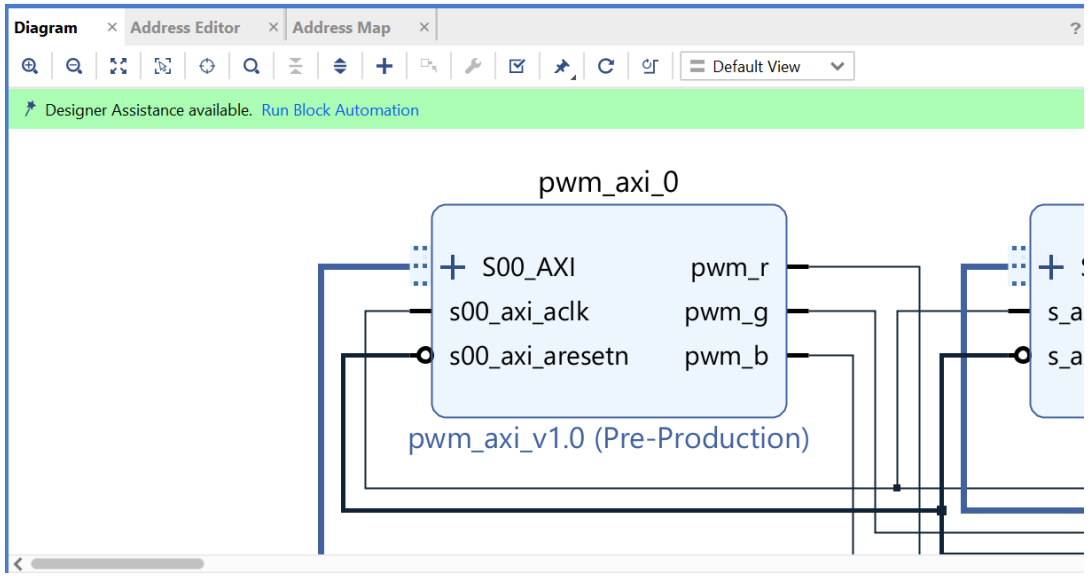




Hint
The display has 8 segment lines and 8 digit enables: 16 output pins. Two channels of one GPIO block are enough, so no extra hardware is needed.

1.5 Add your PWM IP

1. Press + and search for pwm_axi
2. Double-click to add it to the block design
3. Check its ports: the AXI interface and pwm_r, pwm_g, pwm_b

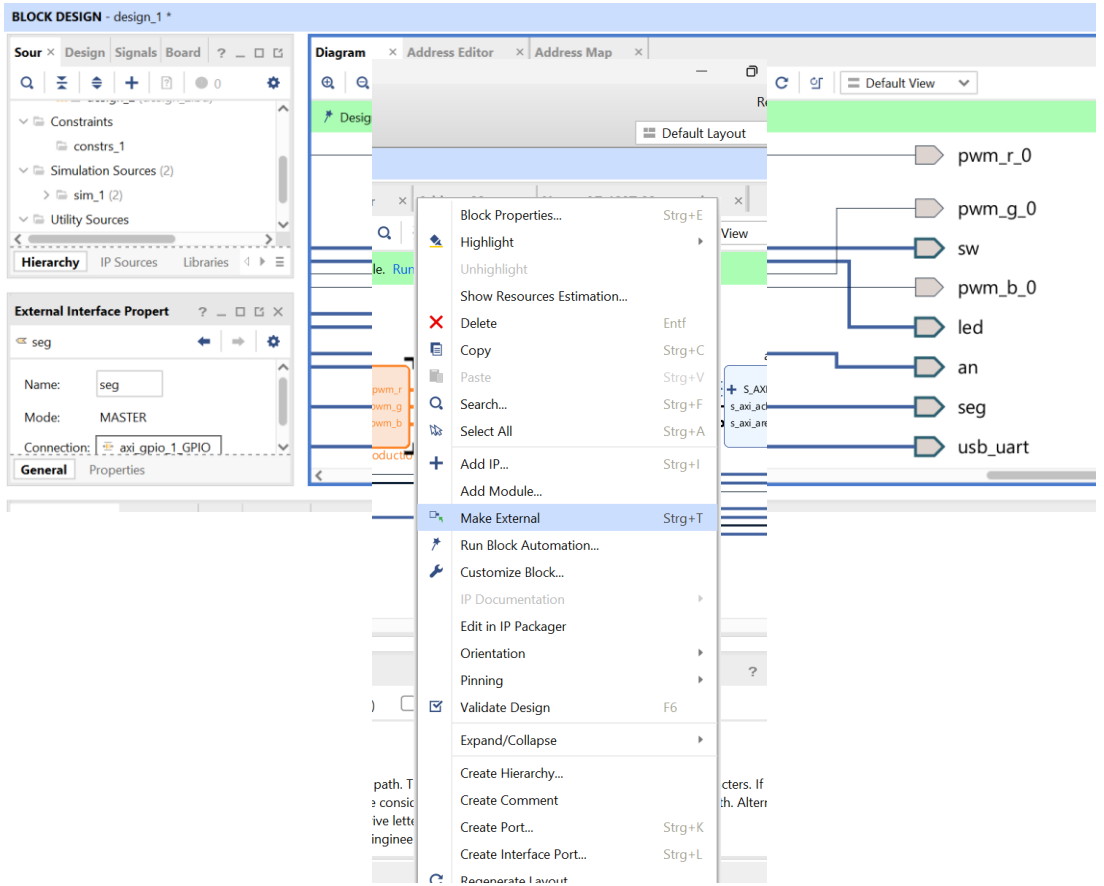


Hint

This is the IP you created in Lab 01. It has an AXI4-Lite interface for the registers and three PWM outputs for the colours of the RGB LED.

1.6 Connect the blocks and create external ports

1. Click Run Connection Automation and select all
2. If Vivado offers board interfaces for the LEDs and switches, select them for GPIO 0
3. Right-click the remaining GPIO outputs: Make External
4. Make the pwm_r, pwm_g and pwm_b outputs external
5. Rename the ports: led, sw, seg, an, pwm_r, pwm_g, pwm_b

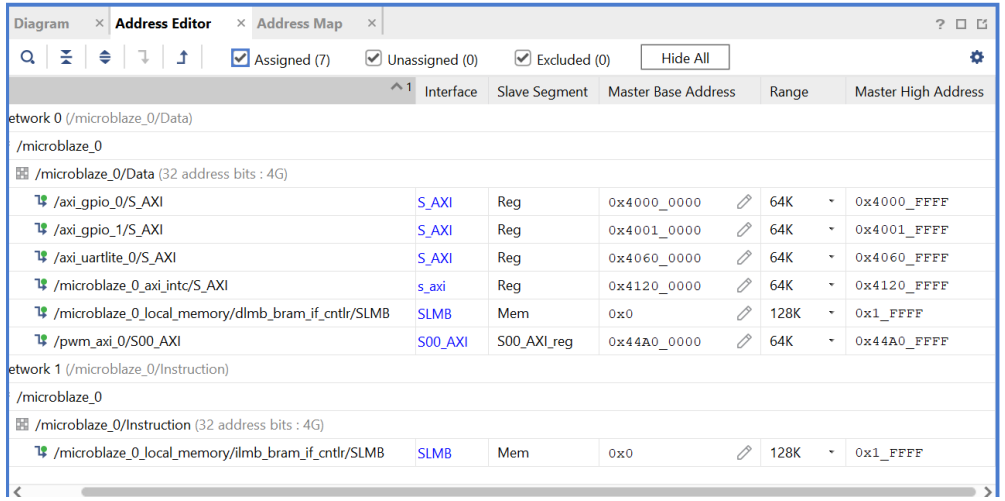


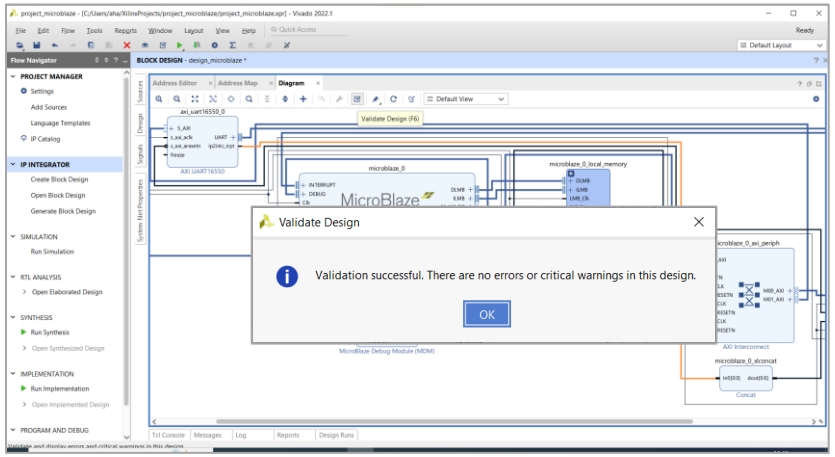
Hint

Connection Automation connects the AXI interfaces, clocks and resets of the new blocks. External ports become the pins of the FPGA: they must get names that you use in the constraints.

1.7 Check the address map and validate

1. Open the Address Editor and check the new address ranges
2. Write down the base addresses of GPIO 0, GPIO 1 and pwm_axi
3. Click Validate Design (F6)
4. Fix errors and run Connection Automation again if needed





Hint
The Address Editor shows where each peripheral lives in the memory map. These addresses end up in xparameters.h and are used by your C code.

1.8 Constraints for LEDs, switches, display and RGB LED

```
## LEDs and switches (copy all 16 each)
set_property -dict {PACKAGE_PIN H17 \
  IOSTANDARD LVCMOS33} [get_ports {led[0]}]
set_property -dict {PACKAGE_PIN J15 \
  IOSTANDARD LVCMOS33} [get_ports {sw[0]}]

## 7-segment: segments CA..CG, DP
set_property -dict {PACKAGE_PIN T10 \
  IOSTANDARD LVCMOS33} [get_ports {seg[0]}]
## 7-segment: digit enables AN0..AN7
set_property -dict {PACKAGE_PIN J17 \
  IOSTANDARD LVCMOS33} [get_ports {an[0]}]
```

```
## RGB LED 16 (verify in master XDC)
set_property -dict {PACKAGE_PIN N15 \
  IOSTANDARD LVCMOS33} [get_ports pwm_r]
set_property -dict {PACKAGE_PIN M16 \
  IOSTANDARD LVCMOS33} [get_ports pwm_g]
set_property -dict {PACKAGE_PIN R12 \
  IOSTANDARD LVCMOS33} [get_ports pwm_b]

## Keep the lines of Lab 02 (clock, reset,
## UART) if no board interface is used.
## Port names must match the HDL wrapper:
## led, sw, seg, an, pwm_r, pwm_g, pwm_b
```

Hint

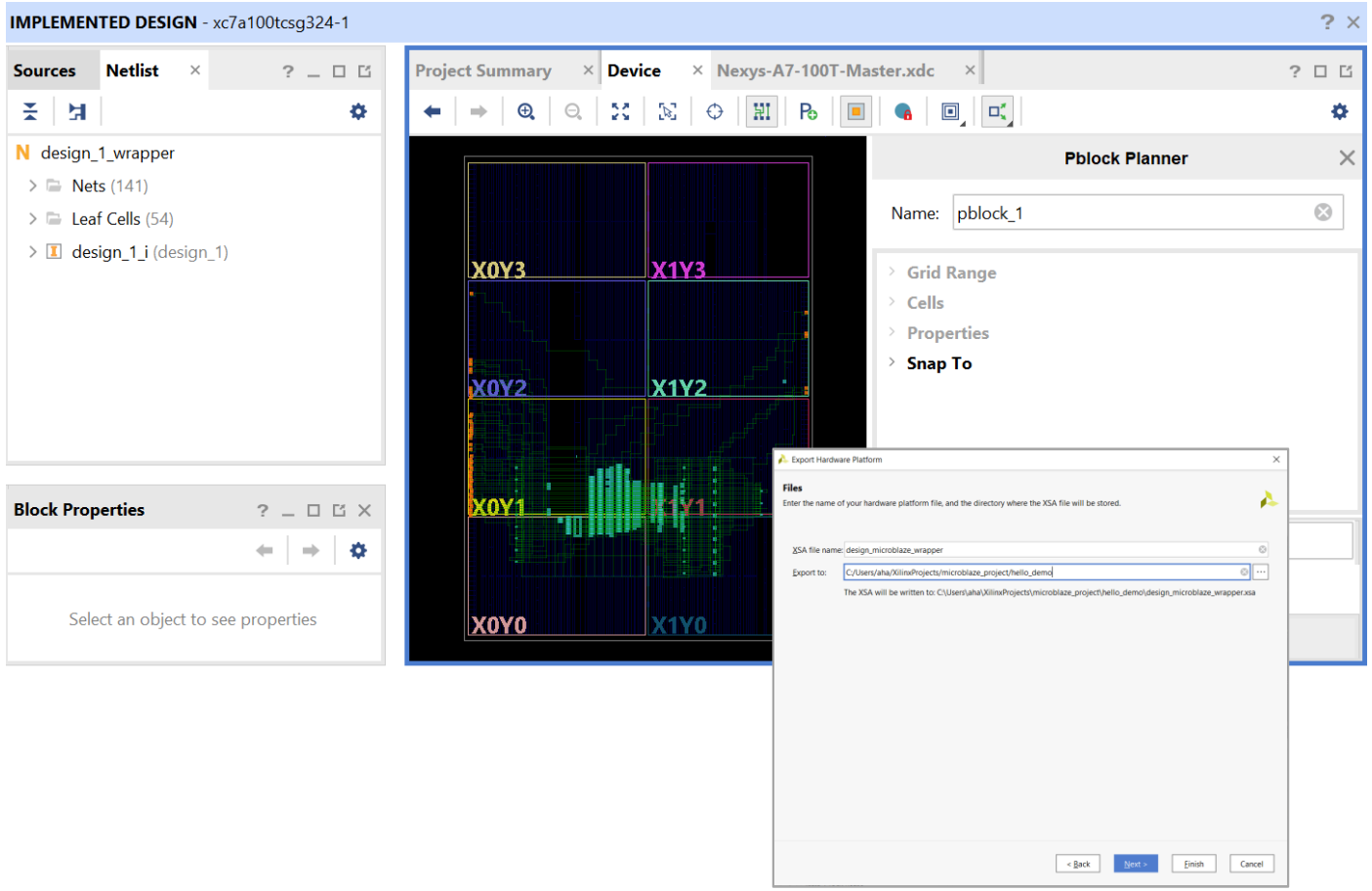
Copy the remaining pins from the Digilent master XDC and rename them to led[n], sw[n], seg[n] and an[n]. Verify every pin on the slide against the master XDC of your board revision. Unconstrained ports stop the bitstream generation.

1.9 Generate the bitstream and export the hardware

1. Update the HDL wrapper if Vivado asks for it
2. Click Generate Bitstream and wait
3. Read the utilization report: compare with Lab 02
4. Export Hardware (include bitstream) and note the XSA location

Hint

The new bitstream contains the processor, the UART, two GPIO blocks and the PWM. The new XSA file describes all of it, including the new addresses.



Task 2: control the hardware from C

1. **Update the Vitis platform with the new XSA** – rebuild the platform, find the new base addresses
2. **Control the LEDs and switches** – read the switches, write the LEDs
3. **Control the 7-segment display** – multiplex the digits in software
4. **Control the RGB LED with the PWM IP** – set the duty registers
5. **Combine everything and test on the board** – one application

Hint
Software controls hardware only through registers at addresses. The same two functions, Xil_In32 and Xil_Out32, are used for GPIO, display and PWM.

2.1 Update the Vitis platform with the new hardware

1. Open the Vitis workspace of Lab 02 or set a new workspace for Lab 03
2. Update the platform with the new XSA file (or create a new platform from it)
3. Build the platform
4. Open xparameters.h and find the base addresses of GPIO 0, GPIO 1 and pwm_axi
5. Create a new application project (Hello World template), e.g. gpio_demo

Hint
The platform contains the address information of the hardware. If it is not updated, the new peripherals do not exist for the software.

2.2 LEDs and switches: read and write registers

```
#include "xil_io.h"
#include "xparameters.h"

#define GPIO0_BASE 0x40000000 /* Address Editor */
#define GPIO_LED   0x08  /* ch 2 data */
#define GPIO_SW    0x00  /* ch 1 data */

int main(void)
{
  while (1) {
    u32 sw = Xil_In32(GPIO0_BASE + GPIO_SW);
    Xil_Out32(GPIO0_BASE + GPIO_LED, sw);
  }
}
```

```
Task:

1. Look up the base address of GPIO 0
   in the Address Editor and put it
   into the define (an example value).
2. Build and run the program.
3. Move the switches: the LEDs follow.

Extend:

4. Show the inverted switch pattern.
5. Light only LED 0 if any switch is up.
```

Hint

Xil_In32 reads a 32-bit register, Xil_Out32 writes one. The offsets are those of the AXI GPIO register map.
Checkpoint: each switch controls the LED above it.

2.3 The 7-segment display: how it works

Digit	Pattern	Digit	Pattern	Digit	Pattern	Digit	Pattern
0	0xC0	4	0x99	8	0x80	C	0xC6
1	0xF9	5	0x92	9	0x90	D	0xA1
2	0xA4	6	0x82	A	0x88	E	0x86
3	0xB0	7	0xF8	B	0x83	F	0x8E

2.4 7-segment display: multiplexing in software

```
#include "xil_io.h"
#include "sleep.h"

#define GPIO0_BASE 0x40000000 /* LEDs, switches */
#define GPIO_LED   0x08       /* ch 2 data */
#define GPIO_SW    0x00       /* ch 1 data */
#define GPIO1_BASE 0x40010000 /* 7-segment */
#define SEG_REG    0x00       /* ch 1: segments */
#define AN_REG     0x08       /* ch 2: enables */

static const u8 seg[16] = {
  0xC0,0xF9,0xA4,0xB0,0x99,0x92,0x82,0xF8,
  0x80,0x90,0x88,0x83,0xC6,0xA1,0x86,0x8E };
```

```
void show_hex(u16 v)
{
  for (int d = 0; d < 4; d++) {
    u32 nib = (v >> (4 * d)) & 0xF;
    Xil_Out32(GPIO1_BASE + AN_REG, 0xFF);
    Xil_Out32(GPIO1_BASE + SEG_REG, seg[nib]);
    Xil_Out32(GPIO1_BASE + AN_REG,
              ~(1u << d) & 0xFF);
    usleep(1000);
  }
}
```

Hint

Digits off, pattern on, digit on, wait 1 ms: repeated for each digit, fast enough that the eye sees all digits at once. Call show_hex() repeatedly. Checkpoint: a hexadecimal number appears on the right four digits.

2.4a The main function: complete the program

```
int main(void)
{
  /* directions: 0 = output */
  Xil_Out32(GPIO1_BASE + 0x4, 0x00);
  Xil_Out32(GPIO1_BASE + 0xC, 0x00);

  while (1) {
    u32 sw = Xil_In32(GPIO0_BASE + GPIO_SW)
             & 0xFFFF;
    Xil_Out32(GPIO0_BASE + GPIO_LED, sw);
    show_hex(sw);
  }
}
```

How to assemble the program

1. One file main.c: the includes, defines, seg table and show_hex from slide 2.4, followed by this main.
2. Set GPIO0_BASE and GPIO1_BASE to the addresses from the Address Editor.
3. First test: replace show_hex(sw) by show_hex(0x1234). You should read 1234 on the four right digits.
4. Without main the linker stops: undefined reference to 'main'.

Hint

Test the display first with a fixed value, then connect it to the switches. A program needs a main function: without it the build fails with undefined reference to 'main'.

2.4b Code explained: registers and patterns

```
#define GPIO1_BASE 0x40010000 /* Address Editor */
#define SEG_REG 0x00   /* ch 1: segments */
#define AN_REG  0x08   /* ch 2: digit enables */

static const u8 seg[16] = {
  0xC0,0xF9,0xA4,0xB0,0x99,0x92,0x82,0xF8,
  0x80,0x90,0x88,0x83,0xC6,0xA1,0x86,0x8E };
```

Addresses

Memory-mapped I/O: address = base address + offset. AXI GPIO registers: 0x00 channel 1 data, 0x04 channel 1 direction, 0x08 channel 2 data, 0x0C channel 2 direction.

Pattern table

seg[n] is the pattern for hex digit n. Bit 0 = segment CA, bit 6 = CG, bit 7 = decimal point. A 0 switches a segment on (active low). Example: 0xC0 = 1100 0000 lights CA to CF: a '0'. 0xF9 = 1111 1001 lights CB and CC: a '1'.

Hint

Each bit of a pattern is one pin of the display. The signals are active low: 0 = on, 1 = off.

2.4c Code explained: show_hex line by line

```
void show_hex(u16 v)
{
  for (int d = 0; d < 4; d++) {
    u32 nib = (v >> (4 * d)) & 0xF;
    Xil_Out32(GPIO1_BASE + AN_REG, 0xFF);
    Xil_Out32(GPIO1_BASE + SEG_REG, seg[nib]);
    Xil_Out32(GPIO1_BASE + AN_REG,
              ~(1u << d) & 0xFF);
    usleep(1000);
  }
}
```

For each digit d = 0 to 3 (0 = rightmost)

nib picks one hex digit. Example v = 0x1234: d = 0 gives 4, d = 1 gives 3, d = 2 gives 2, d = 3 gives 1.

1. All digit enables off (active low: 0xFF).
2. Put the digit's pattern on the shared segment lines.
3. Enable only digit d: ~(1 << d) & 0xFF. Example d = 2 gives 0xFB.
4. Wait 1 ms, then the next digit.

Hint

Multiplexing: one digit is lit at a time for 1 ms. A full pass takes about 4 ms, so each digit is refreshed about 250 times per second and the eye sees a steady display.

Thank you for your attention!

Prof. Dr. -Ing. Ali Hayek

University of Applied Sciences Hamm-Lippstadt (HSHL)

Tel.: **+49 (2381) 8789 – 924**

E-Mail.: **ali.hayek@hshl.de**

2.4d Code explained: main loop and start-up state

```
int main(void)
{
  /* directions: 0 = output */
  Xil_Out32(GPIO1_BASE + 0x4, 0x00);
  Xil_Out32(GPIO1_BASE + 0xC, 0x00);

  while (1) {
    u32 sw = Xil_In32(GPIO0_BASE + GPIO_SW)
             & 0xFFFF;
    Xil_Out32(GPIO0_BASE + GPIO_LED, sw);
    show_hex(sw);
  }
}
```

main

Direction registers: 0 = output. Without outputs the pins stay high-impedance.

while (1): an embedded program never ends. Each pass reads the 16 switches (Xil_In32, & 0xFFFF), writes them to the LEDs (Xil_Out32) and calls show_hex, which takes about 4 ms.

Why are all segments on before the program runs?

After configuration the GPIO outputs are 0. With active-low signals a 0 means on.

Hint

Exercises: 1) show_hex(0xABCD): predict the display. 2) usleep(1000) to 5000 and to 100: what do you see? 3) Light only the decimal point of digit 0. 4) Extend to eight digits.

2.5 Control the RGB LED with your PWM IP

```
#include "xil_io.h"
#include "sleep.h"
#define PWM_BASE 0x44A00000 /* Address Editor */
#define PWM_CTRL 0x00
#define PWM_R    0x04
#define PWM_G    0x08
#define PWM_B    0x0C
int main(void) {
  Xil_Out32(PWM_BASE + PWM_CTRL, 1);
  Xil_Out32(PWM_BASE + PWM_B, 128);
  while (1) {
    for (int v = 0; v < 256; v++) {
      Xil_Out32(PWM_BASE + PWM_R, v);
      Xil_Out32(PWM_BASE + PWM_G, 255 - v);
      usleep(5000);
    }
  }
}
```

```
Task:

1. Find the base address of the PWM
   IP in the Address Editor.
2. Enable the PWM (CTRL = 1).
3. Run the fade and watch the RGB LED.
   Use it as the only content of main.c

Think:

4. Which bits of the registers are used?
   (see the register map of Lab 01)
5. What happens at duty 0 and 255?
```

Hint

Your own IP is controlled exactly like the catalog IP: registers at addresses. This is the benefit of a standard AXI4-Lite interface and a documented register map. Checkpoint: the RGB LED fades.

2.6 Final application: switches control LEDs, display and colour

```
int main(void)
{
  Xil_Out32(PWM_BASE + PWM_CTRL, 1);
  while (1) {
    u32 sw = Xil_In32(GPIO0_BASE + GPIO_SW)
             & 0xFFFF;
    Xil_Out32(GPIO0_BASE + GPIO_LED, sw);
    Xil_Out32(PWM_BASE + PWM_R,
              ((sw >> 0) & 0xF) * 17);
    Xil_Out32(PWM_BASE + PWM_G,
              ((sw >> 4) & 0xF) * 17);
    Xil_Out32(PWM_BASE + PWM_B,
              ((sw >> 8) & 0xF) * 17);
    show_hex(sw);
  }
}
```

```
Behavior:

- LEDs mirror the 16 switches.
- Switches 3..0: red level (0 to 15).
- Switches 7..4: green level.
- Switches 11..8: blue level.
- The 7-segment display shows the
  switch value in hexadecimal.

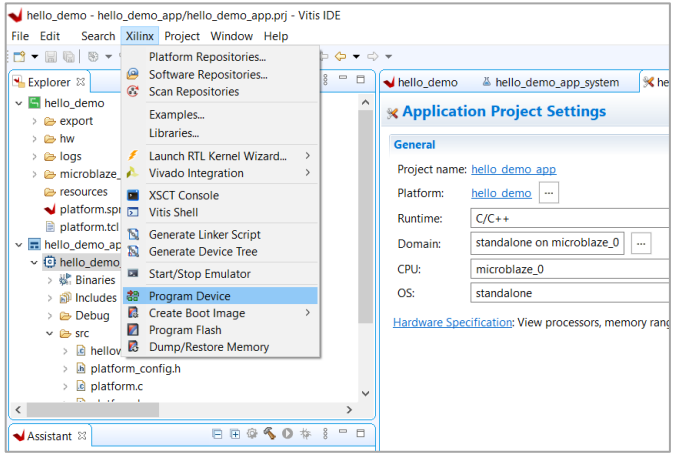
Put the includes and defines (slides 2.4, 2.5),
the seg table, show_hex and this main into ONE
main.c.
```

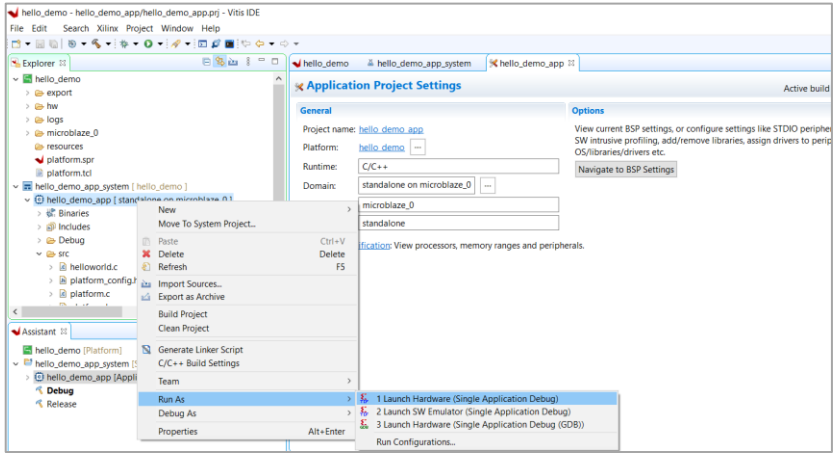
Hint

Input (switches), processing (software) and three outputs (LEDs, display, RGB LED) in one loop. Multiplying by 17 maps the range 0..15 to 0..255. Checkpoint: everything reacts to the switches.

2.7 Program the FPGA and run the application

1. Connect the board and switch it on
2. Program the FPGA with the new bitstream (Program Device)
3. Right-click the application: Run As, Launch on Hardware
4. Move the switches: LEDs, display and RGB LED react





Hint

The FPGA must contain the new hardware before the program is started. An old bitstream does not have the new peripherals and the first access to them hangs.

Additional Tasks…

Counter

Show a running counter on the display; mirror the switches inverted; blink an LED; print the switch value over the UART.

Breathing RGB LED

Let the RGB LED fade in and out smoothly (breathing effect); select the colour with the switches.

UART Commands

Receive colour commands over the UART and set the RGB LED; advanced: use GPIO interrupts instead of polling.

Deliverables and summary

Item	Content
Block design	Screenshot with the GPIO blocks and the PWM IP connected
Reports	Utilization of the system; compare with Lab 02
Software	C source with comments on the register accesses
Demo	Switches control LEDs, display and RGB LED on the Nexys A7
Summary	Memory-mapped I/O, AXI GPIO, multiplexed display, reuse of own IP